

DSA0502 – Query Processing for Data Science

CO3 AT1 – DATA CLEANING CHALLENGE

Course Faculty: Dr. B .SHYAMALA GOWRI

Reg. No.: 192324164

Student Name: Sweety Roselin A

Date: 08/08/2026

Question 3: Banking Loan Approval Dataset

Customer ID	Annual Income	Credit Score	Loan Amount	Employment Status	Age	Existing Debt	Loan Status
L001	750000	720	300000	Full Time	35	50000	Approved
L002	900K	680	400000	FT	42	100000	Approved
L003	1200000	NULL	600000	fulltime	28	200000	Approved
L004	150000	450	-200000	Part Time	30	50000	Rejected
L005	5000000	790	1000000	Self Employed	16	10000	Approved
L006	450000	350	250000	Unemployed	40	300000	Rejected
L007	75000000	800	500000	Full Time	45	100000	Approved
L008	650000	690	350000	FT	33	70000	Approved
L008	650000	690	350000	FT	33	70000	Approved
L010	NULL	710	300000	Full Time	37	80000	Approved

A bank uses customer data to determine loan eligibility. The dataset contains errors that may affect risk assessment and approval decisions.

Data Issues

- Missing credit scores
- Income values recorded in different units
- Negative loan amounts
- Duplicate customer IDs
- Age values below 18
- Employment status entered differently (FT, Full Time, fulltime)
- Extreme outliers in income

Task 1: Identify Critical Issues That Could Impact Loan Decisions (5 Marks)

The dataset contains several errors that can directly affect loan eligibility and credit-risk assessment.

Issue	Customer ID	Problem	Impact
Missing credit score	L003	Credit Score = NULL	Risk assessment becomes incomplete
Different income format	L002	Income = 900K	Can cause incorrect income calculations
Missing income	L010	Annual Income = NULL	Repayment capacity cannot be determined
Negative loan amount	L004	Loan Amount = -200000	Invalid financial value
Age below 18	L005	Age = 16	Invalid for normal adult loan approval
Duplicate customer	L008	Same record occurs twice	Distorts loan statistics
Inconsistent employment	L002, L003, L008	FT, fulltime, Full Time	Creates inconsistent categories
Extreme income	L007	₹75,000,000	Can distort financial analysis

Task 2: Propose Cleaning Techniques for Missing and Inconsistent Data (5 Marks)

Problem	Cleaning Technique
Missing credit score	Verify from the credit bureau; for analysis, use median imputation with a missing-value flag
Missing income	Verify from original financial records
900K income	Convert to ₹900,000
Negative loan amount	Verify and correct from source; otherwise exclude
Age = 16	Verify age and exclude from adult-loan analysis if confirmed
Duplicate L008	Remove duplicate after verification
FT	Standardize to Full Time
fulltime	Standardize to Full Time
Extreme income	Detect using IQR and verify before removing

Example Python Code

```
import pandas as pd

df = pd.read_csv("loan_dataset.csv")

print(df)

df["Annual_Income"] = (
    df["Annual_Income"]
    .astype(str)
    .str.replace("K", "000", regex=False)
)

df["Annual_Income"] = pd.to_numeric(
    df["Annual_Income"],
    errors="coerce"
)

# Standardize employment status
df["Employment_Status"] = df["Employment_Status"].replace({
    "FT": "Full Time",
    "fulltime": "Full Time"
})

# Remove duplicate customer records
df = df.drop_duplicates(
    subset="Customer_ID",
    keep="first"
)
```

```
... Customer_ID Annual_Income Credit_Score Loan_Amount Employment_Status Age \
0      1001      750000      720.0      300000      Full Time      35
1      1002      900K      680.0      400000      FT      42
2      1003      1200000      NaN      600000      fulltime      28
3      1004      150000      450.0      -200000      Part Time      30
4      1005      5000000      790.0      1000000      Self Employed      16
5      1006      450000      350.0      250000      Unemployed      40
6      1007      75000000      800.0      500000      Full Time      45
7      1008      650000      690.0      350000      FT      33
8      1008      650000      690.0      350000      FT      33
9      1010      NaN      710.0      300000      Full Time      37

Existing_Debt Loan_Status
0      50000      Approved
1      100000      Approved
2      200000      Approved
3      50000      Rejected
4      10000      Approved
5      300000      Rejected
6      100000      Approved
7      70000      Approved
8      70000      Approved
9      80000      Approved
```

```
print("Cleaned Dataset:")
```

```
display(df)
```

```
print("Cleaned Dataset:")
display(df)
```

... Cleaned Dataset:

	Customer_ID	Annual_Income	Credit_Score	Loan_Amount	Employment_Status	Age	Existing_Debt	Loan_Status
0	L001	750000.0	720.0	300000	Full Time	35	50000	Approved
1	L002	900000.0	680.0	400000	Full Time	42	100000	Approved
2	L003	1200000.0	NaN	600000	Full Time	28	200000	Approved
3	L004	150000.0	450.0	-200000	Part Time	30	50000	Rejected
4	L005	5000000.0	790.0	1000000	Self Employed	16	10000	Approved
5	L006	450000.0	350.0	250000	Unemployed	40	300000	Rejected
6	L007	75000000.0	800.0	500000	Full Time	45	100000	Approved
7	L008	650000.0	690.0	350000	Full Time	33	70000	Approved
9	L010	NaN	710.0	300000	Full Time	37	80000	Approved

Task 3: Detect and Justify Treatment of Outliers (5 Marks)

The major outlier is **L007**, whose annual income is ₹75,000,000.

IQR Method

```
Q1 = df["AnnualIncome"].quantile(0.25)
```

```
Q3 = df["AnnualIncome"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
outliers = df[
```

```
    (df["AnnualIncome"] < lower) |
```

```
    (df["AnnualIncome"] > upper)
```

```
]
```

```
print(outliers)
```

Outlier Treatment

	Customer_ID	Annual_Income	Credit_Score	Loan_Amount	Employment_Status	\
4	L005	5000000.0	790.0	1000000	Self Employed	
6	L007	75000000.0	800.0	500000	Full Time	
	Age	Existing_Debt	Loan_Status			
4	16	10000	Approved			
6	45	100000	Approved			

L007 should **not automatically be deleted**. A high income may be genuine.

The correct process is:

Detect → Flag → Verify → Correct or Retain

If the income is genuine, it should remain in the dataset.

Task 4: Analyze the Effect of Cleaning on Loan Approval Rates (5 Marks)

Before Cleaning

Total records = 10

Approved = 8

Rejected = 2

Formula:

$$\text{Approval Rate} = \frac{\text{Number of Approved Loans}}{\text{Total Number of Loans}} \times 100$$
$$\text{Approval Rate} = \frac{8}{10} \times 100 = 80\%$$

After Removing Duplicate L008

Total records = 9

Approved = 7

Rejected = 2

Formula:

$$\text{Approval Rate} = \frac{7}{9} \times 100 = 77.78\%$$

After Removing Invalid Records

If L004 and L005 are excluded:

- Approved = 5
- Rejected = 1
- Total = 6

$$\text{Approval Rate} = \frac{5}{6} \times 100$$

83.33%

Dataset Stage	Approved	Rejected	Total	Approval Rate
Before cleaning	8	2	10	80.00%
After duplicate removal	7	2	9	77.78%
After removing invalid records	5	1	6	83.33%

Task 5: Discuss Ethical Implications of Incorrect Data Cleaning Decisions (5 Marks)

Ethical Issue	Explanation
---------------	-------------

Fairness	Incorrect cleaning may unfairly reject eligible customers.
Bias	Removing certain customer records can introduce bias into loan decisions.
Transparency	All data corrections and imputations should be documented.
Privacy	Customer financial information must be protected.
Accuracy	Incorrectly filling missing values can lead to wrong loan decisions.
Human Review	Unusual cases should be verified instead of automatically rejected.
Accountability	Banks should be able to explain how customer data affected the decision.
Auditability	Original and cleaned data should be maintained separately.

Question 4: Smart City Traffic Monitoring Dataset

Sensor ID	Timestamp	Vehicle Count	Average Speed	Traffic Signal Status	Road Name	Weather Condition
S101	01-01-2025 08:00	120	45	Green	Anna Salai	Sunny
S102	01-01-2025 08:05	135	52	Red	anna salai	Sunny
S103	01-01-2025 08:10	-50	48	Green	ANNA SALAI	Cloudy
S104	01-01-2025 08:15	140	350	Yellow	GST Road	Rainy
S105	NULL	110	42	Green	GST ROAD	Sunny
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S107	01-01-2025 08:30	150	55	Green	Old Mahabalipuram Road	Sunny
S10A	01-01-2025 08:35	145	53	Green	OMR	Cloudy
S109	01-01-2025 08:40	138	50	Green	GST Road	Rainy

Data Issues

- Missing timestamps
- Duplicate sensor readings
- Vehicle counts recorded as negative numbers
- Average speed exceeding 300 km/h
- Missing weather information
- Inconsistent road naming conventions
- Sensor IDs with formatting errors

Task 1: Assess the Quality of the Traffic Dataset (5 Marks)

The dataset contains **10 records** and 8 attributes: Sensor ID, Timestamp, Vehicle Count, Average Speed, Traffic Signal Status, Road Name, and Weather Condition.

Data Issue	Example	Impact
Missing timestamp	S105 → NULL	Time-series analysis becomes inaccurate
Negative vehicle count	S103 → -50	Physically impossible value
Extremely high speed	S104 → 350 km/h	Indicates faulty sensor reading
Missing weather	S106 → NULL	Reduces accuracy of weather-based analysis
Duplicate reading	S106 appears twice	Can artificially increase traffic volume
Invalid Sensor ID	S10A	Sensor ID format is inconsistent
Inconsistent road names	Anna Salai, anna salai, ANNA SALAI	Creates duplicate road categories
Inconsistent spaces	GST Road	Affects grouping and analysis
Different road names	OMR, Old Mahabalipuram Road	Same road may be treated as different roads

Task 2: Recommend Cleaning Techniques for Time-Series Data (5 Marks)

Problem	Cleaning Technique
Missing timestamp	Verify from sensor logs; if unavailable, exclude the record
Negative vehicle count	Verify sensor; replace with correct value or mark as invalid
Speed = 350 km/h	Flag as an outlier and verify sensor
Missing weather	Fill using nearby time observations or weather-source data
Duplicate S106	Remove duplicate after verification
Inconsistent timestamps	Convert all timestamps to standard datetime format
Inconsistent road names	Standardize road names

Invalid sensor ID S10A	Verify and correct sensor ID
Missing time-series observations	Use interpolation only when appropriate
Outliers	Detect using domain limits and statistical methods

Example Python Code

```
import pandas as pd

# Convert timestamp to datetime
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="coerce",
    dayfirst=True
)

# Replace negative vehicle counts with missing values
df.loc[df["Vehicle_Count"] < 0, "Vehicle_Count"] = None

# Flag unrealistic speeds
df["Speed_Fault"] = df["Average_Speed"] > 120

# Standardize road names
df["Road_Name"] = (
    df["Road_Name"]
    .astype(str)
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
    .str.title()
)

# Standardize known road names
df["Road_Name"] = df["Road_Name"].replace({
    "Anna Salai": "Anna Salai",
    "Gst Road": "GST Road",
    "Omr": "OMR",
    "Old Mahabalipuram Road": "OMR"
```

```

}))

# Remove duplicate sensor readings
df = df.drop_duplicates(
    subset=["Sensor_ID", "Timestamp"],
    keep="first"
)

```

Sensor_ID	Timestamp	Vehicle_Count	Average_Speed	Traffic_Signal_Status	Road_Name	Weather_Condition	Invalid_Vehicle_Count	Invalid_Speed	Invalid_Sensor_ID	Duplicate
S103	2025-01-01 08:10:00	-50	48	Green	ANNA SALAI	Cloudy	True	False	False	False
S104	2025-01-01 08:15:00	140	350	Yellow	GST Road	Rainy	False	True	False	False
S106	2025-01-01 08:25:00	130	46	Red	OMR	NaN	False	False	False	True
S106	2025-01-01 08:25:00	130	46	Red	OMR	NaN	False	False	False	True
S10A	2025-01-01 08:35:00	145	53	Green	OMR	Cloudy	False	False	True	False

Task 3: Design Rules to Identify Faulty Sensor Readings (5 Marks)

The following validation rules can automatically identify faulty readings:

Rule 1 – Vehicle Count

Vehicle Count < 0 → Faulty

Example:

S103 → **-50** → Faulty reading.

Rule 2 – Average Speed

Average Speed < 0 OR Average Speed > 120 km/h → Faulty

Example:

S104 → **350 km/h** → Faulty/outlier.

Rule 3 – Timestamp

Timestamp is NULL or invalid → Faulty

Example:

S105 → **NULL** → Missing timestamp.

Rule 4 – Sensor ID

Sensor ID must follow the format S + 3 digits.

Example:

S101 ✓

S102 ✓

S10A ✗

Rule 5 – Duplicate Reading

Same Sensor ID + Same Timestamp → Duplicate

Example:

S106 at 08:25 appears twice → Duplicate.

Rule 6 – Traffic Signal Status

Valid values should be:

Green, Yellow, Red

Any other value should be flagged.

Rule 7 – Missing Weather

Weather Condition = NULL → Missing value

Example:

S106 → NULL weather.

Summary

The sensor validation process should be:

Detect → Flag → Verify → Correct/Remove

Faulty readings should **not automatically be deleted** because they may contain useful information about sensor failures.

Task 4: Evaluate How Cleaning Affects Congestion Analysis (5 Marks)

Traffic congestion is generally analyzed using **vehicle count, speed, road, and time**.

The uncleaned dataset can produce misleading results.

Before Cleaning

For example:

- S103 has **-50 vehicles**, which is impossible.
- S104 has **350 km/h**, which can make the road appear to have extremely high traffic speed.
- S106 appears twice, which can increase the calculated traffic volume.
- Anna Salai appears as three different names:
 - Anna Salai

- anna salai
- ANNA SALAI

Therefore, congestion statistics may be incorrect.

After Cleaning

Issue	Before Cleaning	After Cleaning
Negative vehicle count	-50	Flagged/corrected
Speed	350 km/h	Flagged as faulty
Duplicate S106	Counted twice	Counted once
Anna Salai	3 different names	One standardized name
GST Road	Multiple formats	Standardized
OMR	OMR and Old Mahabalipuram Road	Standardized to OMR
Missing weather	NULL	Imputed/verified

Effect on Congestion Analysis

Cleaning can:

1. **Improve average speed calculations**
2. **Improve vehicle-count estimates**
3. **Prevent duplicate records from exaggerating traffic**
4. **Improve road-wise congestion comparison**
5. **Improve time-based congestion trends**

For example, removing the duplicate S106 prevents the same traffic observation from being counted twice.

Task 5: Suggest Automated Monitoring Techniques for Future Data Collection (5 Marks)

Automated monitoring can prevent many data-quality problems before they enter the database.

Technique	Purpose
Real-time validation	Immediately detect invalid readings
Range checks	Reject negative vehicle counts and impossible speeds
Timestamp validation	Ensure every reading has a valid timestamp
Sensor ID validation	Detect incorrectly formatted sensor IDs
Duplicate detection	Identify repeated sensor readings

Automated alerts	Notify administrators when sensors produce abnormal values
Missing-value monitoring	Detect sensors that stop sending data
Data quality dashboard	Display sensor health and data-quality metrics
Automatic logging	Maintain records of sensor failures and corrections
Anomaly detection	Use statistical/ML models to identify unusual traffic patterns

Example Automated Rules

if vehicle_count < 0:

```
    alert("Invalid vehicle count")
```

if average_speed > 120:

```
    alert("Possible faulty speed sensor")
```

if timestamp is None:

```
    alert("Missing timestamp")
```

if duplicate_record:

```
    alert("Duplicate sensor reading")
```

if sensor_id not in valid_sensor_ids:

```
    alert("Invalid sensor ID")
```

*** Ied Dataset:

ensor_ID	Timestamp	Vehicle_Count	Average_Speed	Traffic_Signal_Status	Road_Name	Weather_Condition	Invalid_Vehicle_Count	Invalid_Speed	Invalid_Sensor_ID	Duplicate
S101	2025-01-01 08:00:00	120.0	45.0	Green	Anna Salai	Sunny	False	False	False	False
S102	2025-01-01 08:05:00	135.0	52.0	Red	Anna Salai	Sunny	False	False	False	False
S103	2025-01-01 08:10:00	NaN	48.0	Green	Anna Salai	Cloudy	True	False	False	False
S104	2025-01-01 08:15:00	140.0	NaN	Yellow	GST Road	Rainy	False	True	False	False
S105	NaT	110.0	42.0	Green	GST Road	Sunny	False	False	False	False
S106	2025-01-01 08:25:00	130.0	46.0	Red	OMR	NaN	False	False	False	True